

Devoir IA et Contraintes : Première partie -

Analyse du jeu Escampe

1. Modélisation d'un état du jeu

Pour représenter de manière exhaustive un état du jeu (plateau et pièces restantes), notre modélisation s'appuie sur les éléments suivants :

- **Le plateau de jeu** : Un tableau à deux dimensions de taille 6x6. Chaque case peut contenir un identifiant pour la pièce qui l'occupe : **N** (licorne noire), **n** (paladin noir), **B** (licorne blanche), **b** (paladin blanc), ou - (case vide).
- **L'historique immédiat (la contrainte de liseré)** : Une variable stockant le type de liseré (simple, double ou triple) de la case d'arrivée du dernier coup joué par l'adversaire, car cela dicte les pièces autorisées pour le coup actuel.
- **Le tour de jeu** : Un booléen ou un entier indiquant à qui c'est le tour de jouer (Blanc ou Noir).

Avantages : Cette représentation sous forme de grille 2D est intuitive, très facile à déboguer (elle correspond exactement au format d'affichage textuel demandé) et permet de vérifier facilement si une case est hors limites ou occupée lors du calcul des trajectoires.

Inconvénients : Le parcours d'un tableau 2D pour chercher les pièces légales à chaque tour est un peu plus lent en temps d'exécution que des structures plus optimisées (comme les *bitboards* utilisés dans les moteurs d'échecs perfectionnés), ce qui peut ralentir légèrement le développement de l'arbre de recherche.

Idée: utiliser un bitboard

2. Détermination d'une fin de partie

Les règles spécifiant que la partie s'achève uniquement par la capture de la licorne adverse (les paladins étant imprenables), il n'est pas nécessaire d'analyser l'intégralité du plateau pour déterminer une fin de jeu.

Pour optimiser cette vérification (et accélérer l'exploration de l'arbre de jeu Minimax / Alpha-Bêta), notre classe *EscampeBoard* conservera en mémoire les coordonnées précises des deux licornes à tout instant. Ainsi, lors de la génération ou de l'exécution d'un coup avec la méthode *play()*, il suffira de comparer la case d'arrivée du mouvement avec les coordonnées de la licorne adverse. Si elles correspondent, la licorne est capturée. L'état du jeu peut alors simplement basculer un booléen interne *gameOver* à vrai, ce qui permettra à la méthode *gameOver()* de répondre instantanément en temps constant $O(1)$.

3. Paramètres de difficulté et facteur de branchement maximal

Sources de difficulté : La principale complexité d'Escampe réside dans la contrainte de mouvement : le choix de la pièce à jouer est strictement imposé par le liseré de la case d'arrivée du coup précédent de l'adversaire. Il ne s'agit donc pas seulement de chercher le meilleur coup pour soi, mais surtout de calculer sur quelle case on va atterrir pour contraindre l'adversaire à jouer depuis un liseré qui lui est défavorable (ou pour le forcer à passer son tour). De plus, le mouvement orthogonal sans repasser par la même case exige un algorithme de *pathfinding* local.

Facteur de branchement maximal : Le facteur de branchement connaît un pic énorme lors de la toute première phase du jeu (le placement). Pour placer 1 licorne et 5 paladins sur les 12 cases disponibles (les deux premières lignes d'un bord), on a : 12 choix pour la licorne, puis (5 parmi 11)=462 choix pour les paladins, soit $12 \times 462 = 5544$ possibilités de placement initial par joueur. Pendant la partie (mouvement), le facteur de branchement baisse considérablement. Une pièce sur un liseré triple peut faire 3 pas orthogonaux sans revenir en arrière. À chaque pas, elle a au maximum 3 directions (sauf au premier où elle en a 4), soit environ $4 \times 3 \times 3 = 36$ cases d'arrivée théoriques par pièce. S'il y a plusieurs pièces sur le liseré requis (maximum 6), on aurait au grand maximum ≈ 216 coups possibles. Cependant, les obstacles (pièces imprenables) limitent fortement ce nombre en pratique.

4. Coups imparables et victoire à coup sûr

Oui, il existe une mécanique de "coups imparables" inhérente aux règles d'Escampe. Puisqu'un joueur doit obligatoirement démarrer son mouvement depuis une case ayant le même liseré que la case d'arrivée de l'adversaire, un coup imparable consiste à terminer son mouvement sur un type de liseré (par exemple, un liseré triple) sur lequel l'adversaire ne possède actuellement *aucune pièce*. Dans ce cas de figure, l'adversaire se retrouve dans l'incapacité de jouer et doit passer son tour (il envoie "E"). Cela offre au premier joueur la possibilité de jouer un second coup d'affilée avec n'importe quelle pièce de son choix, ce qui permet très souvent de lancer une attaque décisive et fatale sur la licorne adverse, constituant ainsi une victoire inévitable.

5. Critères pour les heuristiques d'estimation (Fonction d'évaluation)

Pour évaluer un plateau non-terminal lors de l'exploration de l'arbre de jeu (avec l'algorithme Minimax ou Alpha-Bêta), nous utiliserons les critères suivants :

1. **La mobilité (Degré de liberté) :** Le nombre de coups légaux disponibles pour le joueur par rapport au nombre de coups de l'adversaire. Plus on a de coups, plus on contrôle la partie.
2. **La distribution sur les liserés (Sécurité d'activation) :** Pénaliser fortement un état où nos pièces sont absentes d'un type de liseré particulier (simple, double ou triple), car cela nous expose au risque de devoir sauter notre tour.
3. **La distance de Manhattan relative aux licornes :** Minimiser la distance entre nos paladins et la licorne adverse (attaque), tout en maximisant la distance entre les paladins adverses et notre propre licorne (défense).

6. Stratégies par phase de jeu

- **Début de partie (Placement) :** La stratégie est purement positionnelle. Il est vital de placer ses pièces de manière équilibrée sur les différents types de liserés (simples, doubles, triples) pour s'assurer de ne jamais passer son tour. Il faut également "cacher" sa licorne derrière une ligne de paladins imprenables pour bloquer les premières attaques orthogonales.

- **Milieu de partie :** Le but est la restriction de l'adversaire. La tactique vise à atterrir systématiquement sur des liserés qui forcent l'adversaire à déplacer des pièces qui protégeaient sa licorne, ou de l'obliger à jouer depuis un liseré où il a très peu d'options.
- **Fin de partie :** Jeu forcé et agressif. L'objectif est d'isoler la licorne adverse pour faire un "mat", en créant une séquence de coups où l'adversaire ne peut plus l'esquiver.

7. Majorant du nombre de coups et gestion du temps imparti

Majorant du nombre de coups : Les règles spécifient que seules les licornes peuvent être prises (et cela met fin à la partie), les paladins étant imprenables. Comme aucune pièce ne disparaît du plateau pour faire avancer l'état vers une fin inéluctable et qu'aucun mouvement n'est irréversible, il est théoriquement possible que la partie boucle indéfiniment si les joueurs répètent les mêmes déplacements. Le jeu n'a donc pas de majorant mathématique strict du nombre de coups (sauf si une règle de répétition de coups est implémentée en tournoi).

Gestion du temps : Le temps étant limité à 300s pour l'intégralité de la partie, nous allons implémenter la technique de **Iterative Deepening** (Approfondissement Itératif) couplée à l'algorithme **Alpha-Bêta**. *Méthode :* Au lieu de lancer une recherche Minimax à une profondeur fixe trop risquée, le programme cherche le meilleur coup à la profondeur 1, puis 2, puis 3, etc. Avant chaque nouvelle itération de profondeur, l'algorithme vérifie le chronomètre. Si le temps alloué pour le coup en cours approche de sa limite sécuritaire (calculée dynamiquement en fonction du temps global restant et du nombre de tours estimés restants), la recherche s'interrompt immédiatement et renvoie le meilleur coup trouvé à la dernière profondeur complétée. L'élagage Alpha-Bêta garantit par ailleurs que l'exploration des noeuds sera suffisamment rapide pour atteindre des profondeurs satisfaisantes.